| 2025/2026 | AP – Projet 3 – Modération – Itération 1 |
|---|---|
| BTS SIO 2SLAM | Auteur : PORTOLLEAU Anaïs & DELAUNAY-GUITTON Benjamin & CHARBONNEAU Tom |
| | Date de rédaction : 02/03/2026 |

# Projet Modération pour R3st0.fr

## Table des matières


**Projet Modération pour R3st0.fr ..... 1**
**Itération 1 – Initialisation complète de l'application JAVA..... 1**
Ticket 1 - Mise en place du Gitlab (labels, jalons, ...) ..... 2
Ticket 2 : Analyse du cahier des charges..... 5
Ticket 3 - Création des classes métiers et DAO ..... 6
Programmes : .....7
Ticket 4 - Affichage de la liste des restaurants de la BDD sur la console..... 8
Programmes : .....8
Test Fonctionnel : .....8
Ticket 5 - Maquette des écrans / Création des interfaces graphiques ..... 8
Programmes : .....8
Test fonctionnel : .....8
Ticket 6 - Analyse de la priorisation des tâches..... 8


Lien du Gitlab : URL de notre dépôt

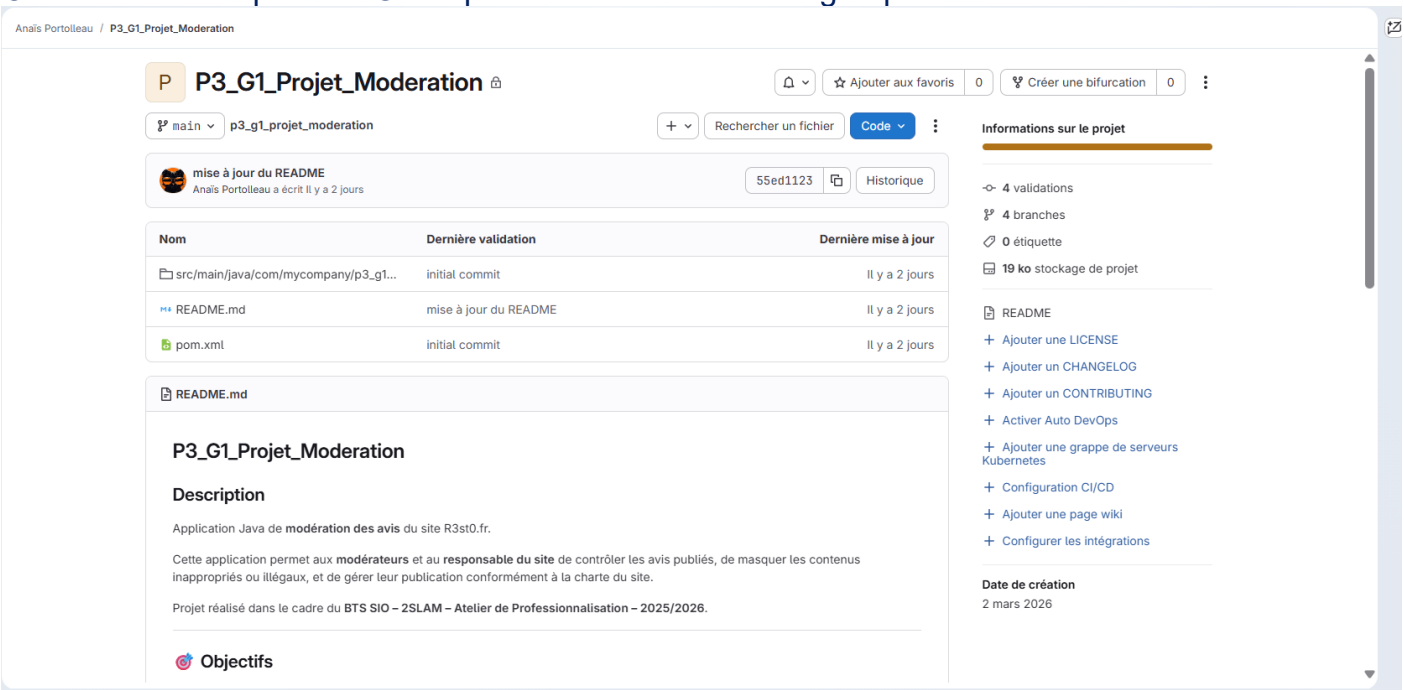
# Itération 1 – Initialisation complète de l'application JAVA

Les **rapports d'itérations** tiendront lieu de documentation, et comporteront :

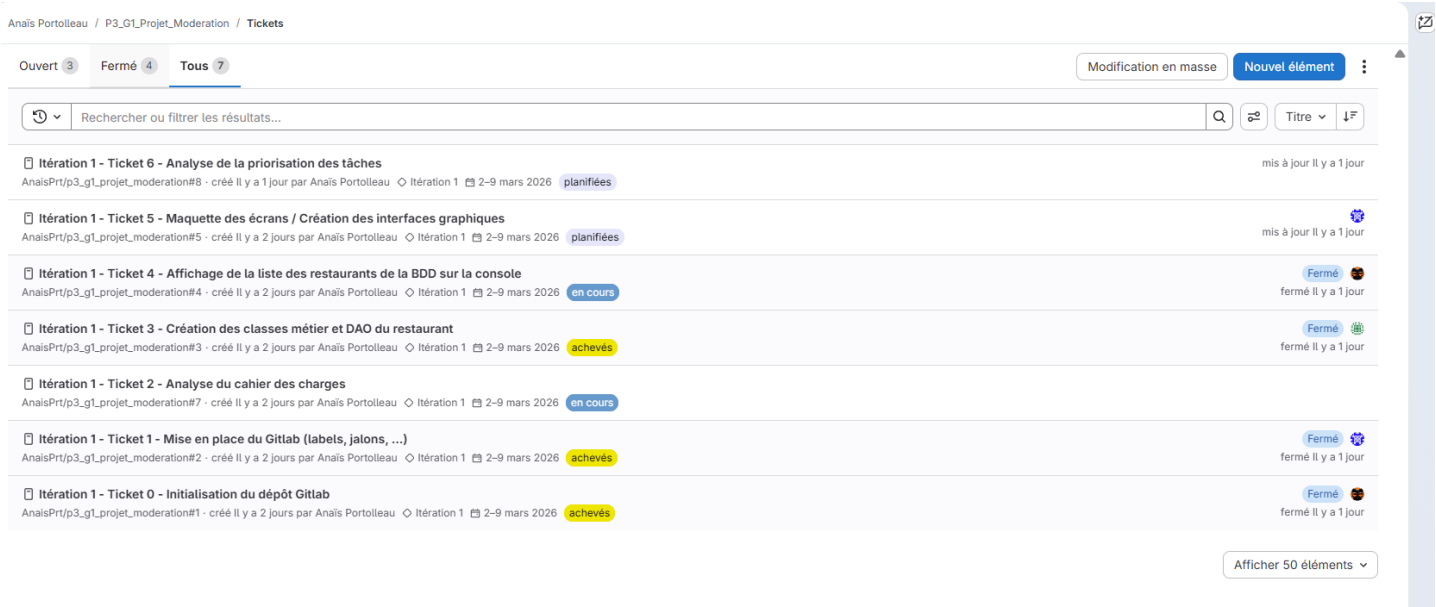
- Une documentation utilisateur : chaque "livrable" [1] doit être accompagné d'un mode opératoire expliquant les opérations d'installation et de mise en œuvre des fonctionnalités disponibles.
- Une documentation technique incluant :
  - les documents relatifs à la gestion de projet :
    - planification prévisionnelle mise à jour (Gantt, …),
    - comptes-rendus datés des daily-scrums de chaque séance,
    - la répartition des tâches : instantané du tableau des tâches mis à jour à l'issue du daily- scrum ,
    - l'adresse du dépôt de code de l'équipe ;
  - Les spécifications et l'analyse, avec la modélisation (diagrammes UML, ...) et les explications associées ;
  - Le code source :
    - le nom de la branche correspondante sur le dépôt de code,
    - les explications sur les évolutions du code source,
    - la documentation du code source (générée par javadoc d'après les commentaires) ;
  - les tests unitaires et d'intégration : présentation des jeux d'essai, traces d'exécutions commentées ;
  - les tests fonctionnels : présentation des jeux d'essai, copies d'écrans commentées ;
  - le bilan (ce qui a été fait, ce qui reste à faire, les difficultés rencontrées).

# Ticket 1 - Mise en place du Gitlab (labels, jalons, ...)

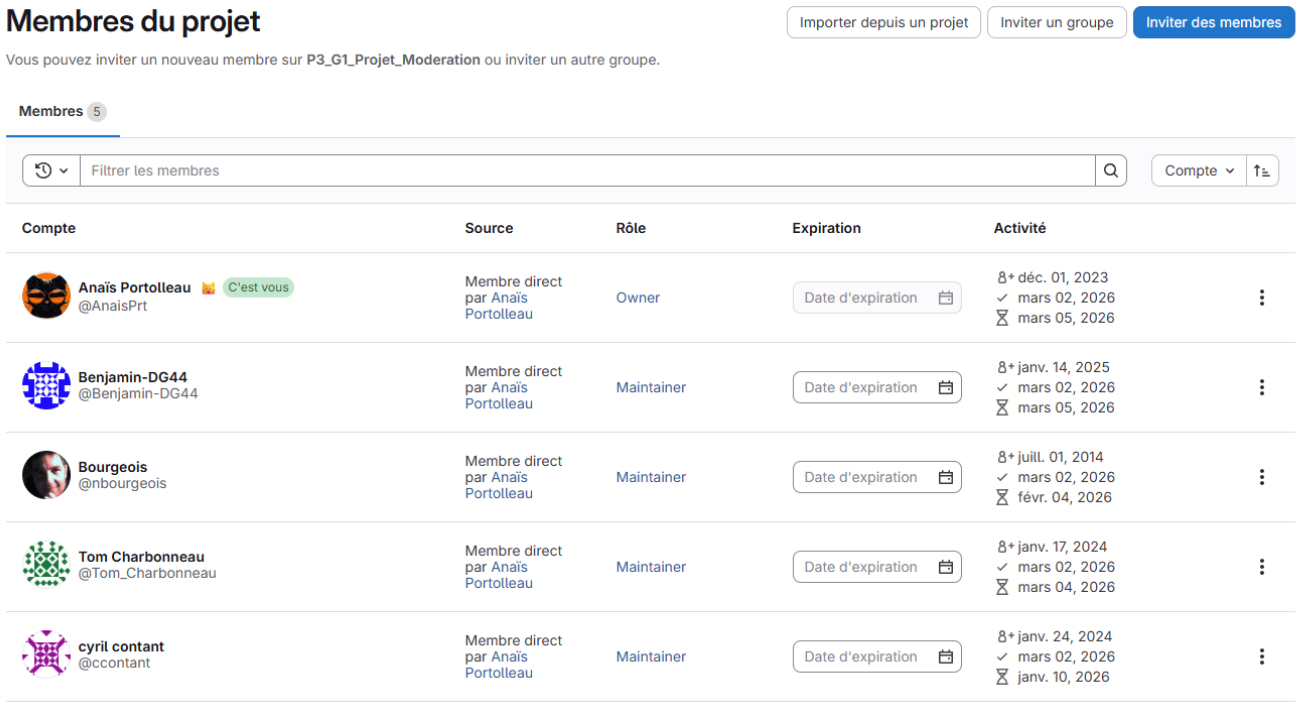
On a bien mis en place le Gitlab pour faciliter le travail de groupe :



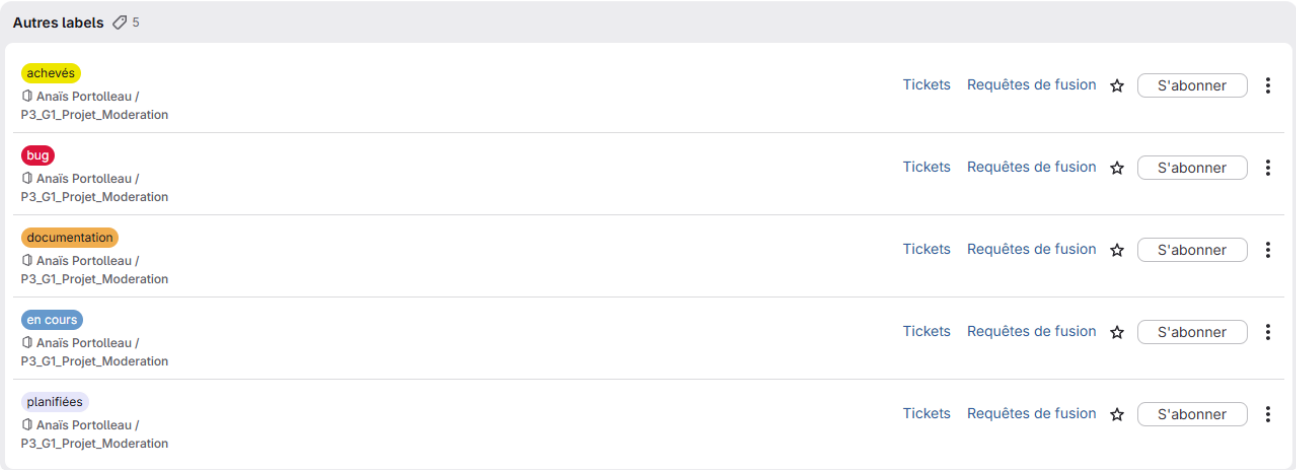

On a créé différents tickets de l'itération 1 :



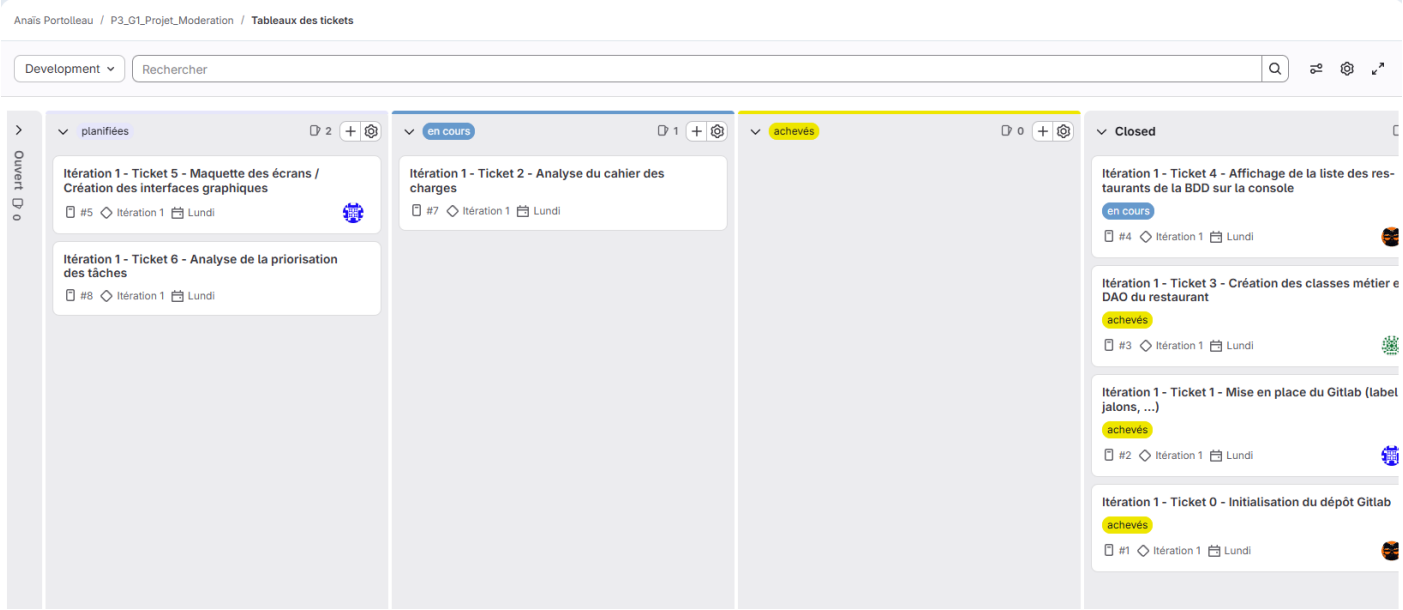
On a ajouté les membres du projet ainsi que les professeurs :



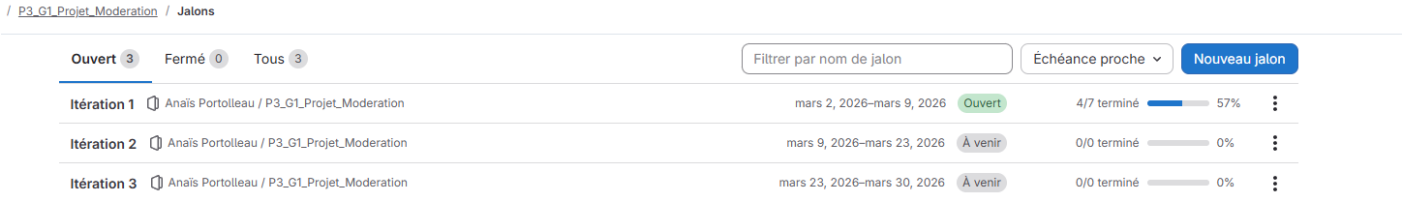

On a créé les différents labels qu'on va pouvoir utiliser :



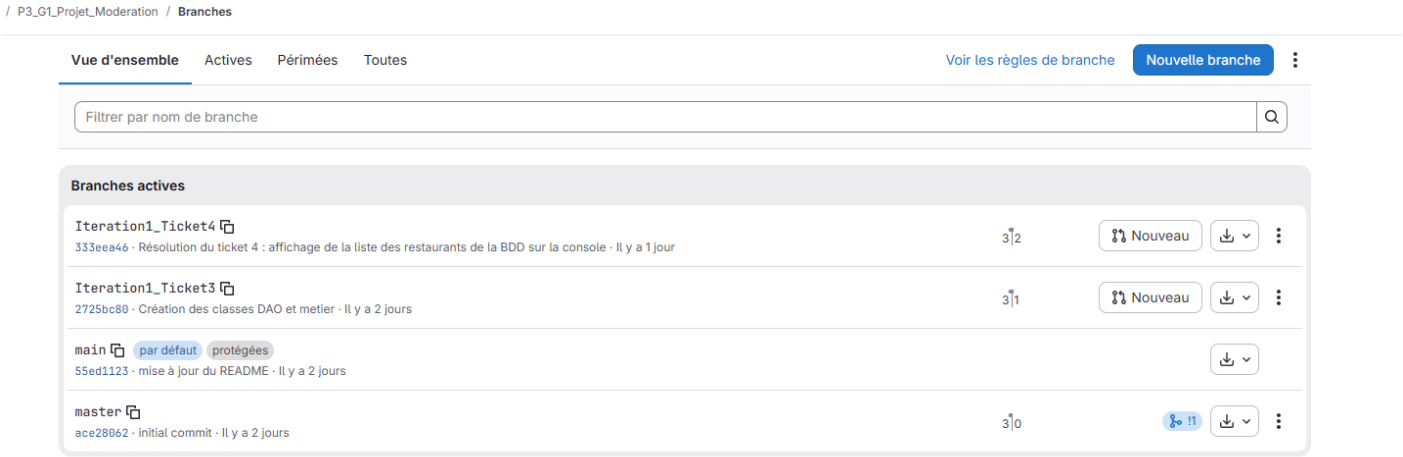
On a organisé le Tableau des tickets :




On a commencé à créer le premier jalon pour l'itération 1 ainsi que la création des suivants :




Et le début des tickets qui est résolu à chaque fois sur une branche précise pour ensuite faciliter la fusion du code et pour éviter les problèmes de merge avec la création d'une branche par ticket :




Lien du Gitlab : URL de notre dépôt

## Ticket 2 : Analyse du cahier des charges

Nous avons bien compris ce qui était attendu globalement pour les prochaines séances. C'est pourquoi nous avons ajoutons le ticket n°6 qui va servir à planifier les prochaines tâches à réaliser.

# Ticket 3 - Création des classes métiers et DAO

Branche utilisée : iteration1_ticket3

Je me suis occupé de développer les différentes classes métiers et les différentes classes DAO afin de permettre à Anaïs de pouvoir les afficher dans la console lors du prochain ticket.

La classe métier Resto a été implémentée pour refléter fidèlement la table resto de la base de données, avec des champs comme : nomR, numAdr, villeR, descR, horairesR, ... mis en attributs pour la classe Resto.

Pour assurer l'accès à ces données, la classe RestoDao a été développée conjointement. Cette classe centralise la logique de persistance et les requêtes SQL spécifiques aux restaurants. Elle propose des méthodes dédiées (comme la récupération de tous les établissements ou encore la recherche par identifiant et bien d'autres fonctions) qui extraient les données brutes de la base et les convertissent directement en objets Resto manipulables par l'application.

## Programmes :

*Resto.java:*

J'ai commencé par développer la classe métier Resto comme on peut voir ci-dessous. Dans cette classe on peut retrouver tous les attributs qui seront utiles à la classe Resto, ainsi que plusieurs possibilités de constructeurs et que ces accesseurs et ces mutateurs associés.

```java
public class Resto {

    private Long idR;
    private String nomR;
    private String numAdrR;
    private String voieAdrR;
    private String cpR;
    private String villeR;
    private Float latitudeDegR;
    private Float longitudeDegR;
    private String descR;
    private String horairesR;

    /**
     * Constructeur vide.
     */
    public Resto() {
    }
```

```java
/**
 * Constructeur pour l'ajout en BDD.
 *
 * @param nomR nom du restaurant.
 * @param numAdrR numéro d'adresse du restaurant.
 * @param voieAdrR voie d'adresse du restaurant.
 * @param cpR code postale du restaurant.
 * @param villeR ville du restaurant.
 * @param latitudeDegR latitude du restaurant.
 * @param longitudeDegR longitude du restaurant.
 * @param descR description du restaurant.
 * @param horairesR horaires d'ouvertures du restaurant.
 */
public Resto(String nomR, String numAdrR, String voieAdrR, String cpR, String villeR,
        Float latitudeDegR, Float longitudeDegR, String descR, String horairesR) {
    this.nomR = nomR;
    this.numAdrR = numAdrR;
    this.voieAdrR = voieAdrR;
    this.cpR = cpR;
    this.villeR = villeR;
    this.latitudeDegR = latitudeDegR;
    this.longitudeDegR = longitudeDegR;
    this.descR = descR;
    this.horairesR = horairesR;
}
```

```java
/**
 * Constructeur complet.
 *
 * @param idR identifiant du restaurant.
 * @param nomR nom du restaurant.
 * @param numAdrR numéro d'adresse du restaurant.
 * @param voieAdrR voie d'adresse du restaurant.
 * @param cpR code postale du restaurant.
 * @param villeR ville du restaurant.
 * @param latitudeDegR latitude du restaurant.
 * @param longitudeDegR longitude du restaurant.
 * @param descR description du restaurant.
 * @param horairesR horaires d'ouvertures du restaurant.
 */
public Resto(Long idR, String nomR, String numAdrR, String voieAdrR, String cpR, String villeR,
        Float latitudeDegR, Float longitudeDegR, String descR, String horairesR) {
    this.idR = idR;
    this.nomR = nomR;
    this.numAdrR = numAdrR;
    this.voieAdrR = voieAdrR;
    this.cpR = cpR;
    this.villeR = villeR;
    this.latitudeDegR = latitudeDegR;
    this.longitudeDegR = longitudeDegR;
    this.descR = descR;
    this.horairesR = horairesR;
}
```

```java
/**
 * Constructeur simple.
 *
 * @param nomR nom du restaurant.
 * @param villeR ville du restaurant.
 */
public Resto(String nomR, String villeR) {
    this.nomR = nomR;
    this.villeR = villeR;
}
```

Puis les accesseurs et mutateurs associés :

```java
// Getters et Setters
public Long getIdR() {
    return idR;
}

public void setIdR(Long idR) {
    this.idR = idR;
}

public String getNomR() {
    return nomR;
}

public void setNomR(String nomR) {
    this.nomR = nomR;
}

public String getNumAdrR() {
    return numAdrR;
}

public void setNumAdrR(String numAdrR) {
    this.numAdrR = numAdrR;
}

public String getVoieAdrR() {
    return voieAdrR;
}
```

*RestoDao.java*:

J'ai également développé la classe DAO associée dont une méthode nommée getAll qui permet d'avoir la liste de tous les restaurants avec cette méthode nommée getAll() :

```java
public class RestoDao {

    /**
     * Récupère tous les restaurants de la BDD.
     *
     * @return une liste d'objets Resto.
     * @throws SQLException erreur SQL.
     * @throws IOException erruer avec la valeur d'entrée ou de sortie.
     */
    public static List<Resto> getAll() throws SQLException, IOException {
        List<Resto> lesResto = new ArrayList<>();
        String query = "SELECT * FROM resto";

        try (Connection cnx = ConnexionBDD.getConnexion(); PreparedStatement pstmt = cnx.prepareStatement(query); ResultSet rs

            while (rs.next()) {
                Resto unResto = new Resto(
                        rs.getLong("idR"),
                        rs.getString("nomR"),
                        rs.getString("numAdrR"),
                        rs.getString("voieAdrR"),
                        rs.getString("cpR"),
                        rs.getString("villeR"),
                        rs.getFloat("latitudeDegR"),
                        rs.getFloat("longitudeDegR"),
                        rs.getString("descR"),
                        rs.getString("horairesR")
                );
                lesResto.add(unResto);
            }
        }
```

Et j'ai repris cette méthode pour les autres fichiers de notre premier projet site resto en PHP où j'ai converti le code en Java, avec les classes DAO ainsi que métier :

- P3_G1_Projet_Moderation [Iteration1_Ticket3]
  - Source Packages
    - modele.dao
      - CritiqueDao.java
      - PhotoDao.java
      - ReservationDao.java
      - RestoDao.java
      - TypeCuisineDao.java
      - UtilisateurDao.java
    - modele.metier
      - Critique.java
      - DBConnection.java
      - Photo.java
      - Reservation.java
      - Resto.java
      - TypeCuisine.java
      - Utilisateur.java

# Ticket 4 - Affichage de la liste des restaurants de la BDD sur la console

Branche utilisée : iteration1_ticket4

## Programmes :

Dans le fichier RestoDao.java, j'ai renommé la méthode findAll en une méthode nommée getAll :

```java
/**
 * Récupère tous les restaurants de la BDD.
 *
 * @return une liste d'objets Resto.
 * @throws SQLException erreur SQL.
 * @throws IOException erruer avec la valeur d'entrée ou de sortie.
 */
public static List<Resto> getAll() throws SQLException, IOException {
    List<Resto> lesResto = new ArrayList<>();
    String query = "SELECT * FROM resto";

    try (Connection cnx = ConnexionBDD.getConnexion(); PreparedStatement pstmt = cnx.prepareStatement(query); ResultSet rs

        while (rs.next()) {
            Resto unResto = new Resto(
                    rs.getLong("idR"),
                    rs.getString("nomR"),
                    rs.getString("numAdrR"),
                    rs.getString("voieAdrR"),
                    rs.getString("cpR"),
                    rs.getString("villeR"),
                    rs.getFloat("latitudeDegR"),
                    rs.getFloat("longitudeDegR"),
                    rs.getString("descR"),
                    rs.getString("horairesR")
            );
            lesResto.add(unResto);
        }
    }

    return lesResto;
}
```

Puis j'ai créé cette méthode getAll() qui récupère tous les restaurants dans la table resto. Elle ouvre une connexion via ConnexionBDD, exécute la requête SQL et parcourt le ResultSet. Pour chaque ligne, elle crée un objet Resto avec les données de la base et l'ajoute à la liste lesResto. Enfin, elle renvoie la liste complète des restaurants.

Ainsi que la création de la méthode getByID qu'on peut voir ci-dessous dans le même fichier :

```java
/**
 * Récupère un restaurant.
 *
 * @param id identifiant du restaurant.
 * @return un objet resto si l'identifiant est dans la BDD, null sinon.
 * @throws SQLException erreur SQL.
 * @throws IOException erruer avec la valeur d'entrée ou de sortie.
 */
public static Resto getById(Long id) throws SQLException, IOException {
    String query = "SELECT * FROM resto WHERE idR = ?";

    try (Connection cnx = ConnexionBDD.getConnexion(); PreparedStatement pstmt = cnx.prepareStatement(query)) {

        pstmt.setLong(1, id);
        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return new Resto(
                        rs.getLong("idR"),
                        rs.getString("nomR"),
                        rs.getString("numAdrR"),
                        rs.getString("voieAdrR"),
                        rs.getString("cpR"),
                        rs.getString("villeR"),
                        rs.getFloat("latitudeDegR"),
                        rs.getFloat("longitudeDegR"),
                        rs.getString("descR"),
                        rs.getString("horairesR")
                );
            }
        }
    }

    return null;
```

Cette méthode permet de récupérer un restaurant dans la base de données à partir de son identifiant. Elle prépare une requête SQL qui sélectionne le restaurant dont l'id correspond au paramètre fourni. La connexion à la base et la requête préparée sont créées dans un bloc try avec ressources pour être fermées automatiquement. L'identifiant est ensuite injecté dans la requête avant son exécution. Si un résultat est trouvé, les informations du restaurant sont lues dans le ResultSet. Un nouvel objet Resto est alors créé avec les données récupérées. Si aucun restaurant ne correspond à l'identifiant, la méthode retourne null.

J'ai créé ensuite un fichier TestRestoDao afin de tester la classe RestoDao déjà présente dans le projet 1. Ce fichier permet de vérifier que les méthodes du DAO fonctionnent correctement et que la récupération des données depuis la base de données fonctionne comme prévu.

Dans ce fichier, j'ai créé une classe contenant une méthode main, ce qui permet d'exécuter directement le programme pour réaliser les tests. Le premier test concerne la méthode getAll(). Cette méthode permet de récupérer tous les restaurants présents dans la base de données. Pour cela, j'appelle RestoDao.getAll() qui retourne une liste d'objets de type Resto. Les résultats sont ensuite stockés dans une variable de type List<Resto> appelée lesRestos.

Une boucle for est ensuite utilisée pour parcourir cette liste et afficher chaque restaurant dans la console avec System.out.println(r). Cela permet de vérifier que les objets Resto sont correctement récupérés et que les informations provenant de la base de données sont bien affichées. À la fin du parcours de la liste, le programme affiche également le nombre total de restaurants récupérés grâce à la méthode lesRestos.size().

Le code est placé dans un bloc try / catch afin de gérer les exceptions éventuelles. Les exceptions SQLException et IOException peuvent se produire lors de l'accès à la base de données ou lors de la lecture de certains fichiers nécessaires à la connexion. Si une erreur survient, un message est affiché dans la console afin d'indiquer l'origine du problème et de faciliter le débogage.

Un second test est prévu pour la méthode getById(), qui doit permettre de récupérer un restaurant spécifique à partir de son identifiant. Cette méthode sert à vérifier que la recherche d'un restaurant précis dans la base de données fonctionne correctement et que l'objet correspondant est bien retourné.

TestRestoDao.java

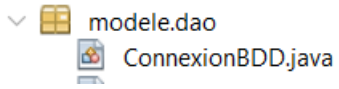
Source History

```java
10  * Classe de test pour la classe RestoDAO.
11  *
12  * @author anais
13  */
14 public class TestRestoDao {
15
16     public static void main(String[] args) {
17
18         // Test 1 : getAll()
19         System.out.println("\nTest 1 : RestoDao.getAll");
20         try {
21             List<Resto> lesRestos = RestoDao.getAll(); // Récupération de tous les restaurants.
22             // Parcourt la liste d'objets Resto lesRestos.
23             for (Resto r : lesRestos) {
24                 System.out.println(r);
25             }
26             System.out.println(lesRestos.size() + " restaurants trouvés"); // Nombre de restaurants récupérés
27
28         } catch (SQLException | IOException ex) {
29             System.out.println("TestRestoDAO - échec getAll : " + ex.getMessage());
               ex.printStackTrace();
31         }
32
33         // Test 2 : getById()
34         System.out.println("\nTest 2 : RestoDao.getById");
35         try {
36             List<Resto> lesRestos = RestoDao.getAll(); // Récupération de tous les restaurants.
37             // Parcourt la liste d'objets Resto lesRestos.
38             for (Resto r : lesRestos) {
39                 System.out.println(r);
40             }
41             System.out.println(lesRestos.size() + " restaurants trouvés"); // Nombre de restaurants récupérés
42
43         } catch (SQLException | IOException ex) {
44             System.out.println("TestRestoDAO - échec getAll : " + ex.getMessage());
               ex.printStackTrace();
46         }
```

Le fichier est défini dans le package test.dao pour bien organiser le projet.

- test.dao
  - TestConnexionBDD.java
  - TestRestoDao.java
- test.metier
  - TestResto.java

J'ai rajouté également un fichier ConnexionBDD. La classe ConnexionBDD centralise la connexion à la base pour éviter de répéter le code partout.




Elle lit les paramètres dans un fichier restoJdbc.properties pour rendre la configuration flexible. Elle charge le driver MySQL et garantit qu'une seule connexion est utilisée.

Avec d'abord la création de la classe ainsi que son constructeur :

```
9   /**
10   * Classe de connexion à la bdd
11   * @author anais
12   */
13  public class ConnexionBDD {
14
15      private static Connection cnx;
16
17      /**
18       * Constructeur
19       */
20      private ConnexionBDD() {
21      }
22
```

Cette méthode getConnexion() a pour objectif de récupérer la connexion à la base de données et de s'assurer qu'elle est disponible avant toute opération. Elle est définie comme static, ce qui signifie qu'elle peut être appelée directement sans créer d'instance de la classe.

Lorsqu'elle est appelée, elle vérifie d'abord si la connexion cnx est nulle ou fermée. Si c'est le cas, elle tente d'établir une nouvelle connexion. Pour cela, elle lit un fichier de configuration restoJdbc.properties qui contient les informations nécessaires à la connexion : l'URL de la base de données, le nom d'utilisateur et le mot de passe.

Le fichier est chargé via un InputStream en utilisant le ClassLoader du thread courant. Si le fichier n'est pas trouvé, la méthode lève une exception SQLException avec un message explicite. Une fois les propriétés chargées, la méthode récupère les valeurs de url, utilBD (utilisateur) et mdpBD (mot de passe). Elle charge ensuite le driver JDBC MySQL avec Class.forName("com.mysql.cj.jdbc.Driver").

Enfin, elle utilise DriverManager.getConnection(url, user, password) pour établir la connexion et l'assigne à la variable cnx. Si tout se passe bien, un message "Connexion BDD OK" est affiché dans la console. Si une erreur survient à n'importe quelle étape, elle est capturée et une SQLException est levée avec le détail de l'erreur.

Ainsi, cette méthode garantit qu'une connexion unique et réutilisable est disponible pour toutes les opérations sur la base de données dans le projet.

```java
    /**
     * méthode de récupération de la connexion
     * à la base de données
     * @return
     * @throws SQLException
     */
    public static Connection getConnexion() throws SQLException {

        if (cnx == null || cnx.isClosed()) {
            try {
                Properties properties = new Properties();
                InputStream input = Thread.currentThread()
                        .getContextClassLoader()
                        .getResourceAsStream("restoJdbc.properties");

                if (input == null) {
                    throw new SQLException("Fichier restoJdbc.properties introuvable");
                }

                properties.load(input);
                String url = properties.getProperty("url");
                String user = properties.getProperty("utilBD");
                String password = properties.getProperty("mdpBD");
                Class.forName("com.mysql.cj.jdbc.Driver");
                cnx = DriverManager.getConnection(url, user, password);
                System.out.println("Connexion BDD OK");

            } catch (Exception e) {
                throw new SQLException("Erreur connexion BDD", e);
            }
        }
        return cnx;
    }
}
```

Et j'ai ajouté son test associé comme on peut le voir ci-dessous :

TestRestoDao.java × TestConnexionBDD.java ×

Source History

```java
package test.dao;

import modele.dao.ConnexionBDD;
import java.sql.Connection;
import java.sql.SQLException;

/**
 * Classe de test pour la classe ConnexionBDD.
 *
 * @author anaïs
 */
public class TestConnexionBDD {

    public static void main(String[] args) throws Exception {
        Connection cnx = null;
        try {
            System.out.println("\n Test de connexion");
            cnx = ConnexionBDD.getConnexion(); // Ouverture de la connexion
            System.out.println("ConnexionBDD : connexion réussie");
            cnx.close(); // Fermeture de la connexion
        } catch (SQLException e) {
            e.printStackTrace();
        }

    }

}
```

Également tester la classe resto créé dans le ticket précédent avec ce fichier de test :

TestRestoDao.java × TestConnexionBDD.java × TestResto.java ×

Source History

```java
package test.metier;

import modele.metier.Resto;

/**
 *
 * @author anais
 */
public class TestResto {

    public static void main(String[] args) {
        System.out.println("TestResto");

        // Test 1 : Création d'un restaurant "simple".
        Resto unResto = new Resto("l'Escale", "Biarritz");
        System.out.println(unResto.toString());
    }
}
```

Et pour finir j'ai ajouté également ce fichier nommé restoJdbc.properties :

```
restoJdbc.properties [-/A]
Source  History
1  ######################################
2  # Paramètres de connexion JDBC
3  # Utilisation : décommenter les paramètres spécifiques au SGBD visé
4  # Les paramètres suivants doivent être renseignés : url, utilBD, mdpBD
5  ######################################
6
7  # MYSQL / MariaDB  local : mysql-connector-java-8.0.18.jar
8  url=jdbc:mysql://localhost:3306/resto2
9  utilBD=root
10 mdpBD=
11
```

Ce fichier restoJdbc.properties a été ajouté pour centraliser et externaliser la configuration de la base de données.

Il permet de stocker l'URL de connexion, le nom d'utilisateur et le mot de passe séparément du code Java, ce qui rend la connexion facile à modifier sans toucher au code. En le chargeant dans ConnexionBDD, on peut établir la connexion JDBC de manière flexible et réutilisable pour toutes les classes qui accèdent à la base de données.

## Test Fonctionnel :

Objectif : tester la connexion à la BDD.

Je vais tester d'abord la connexion pour savoir si on arrive bien jusqu'à la base de données, si on arrive à la joindre. Si on test avec le fichier TestConnexionBDD cela donne ceci :

```
 Test de connexion
getConnexion : jdbc:mysql://localhost:3306/resto2
ConnexionBDD : connexion r►ussie
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  3.581 s
Finished at: 2026-03-03T14:31:18+01:00
------------------------------------------------------------------------
```

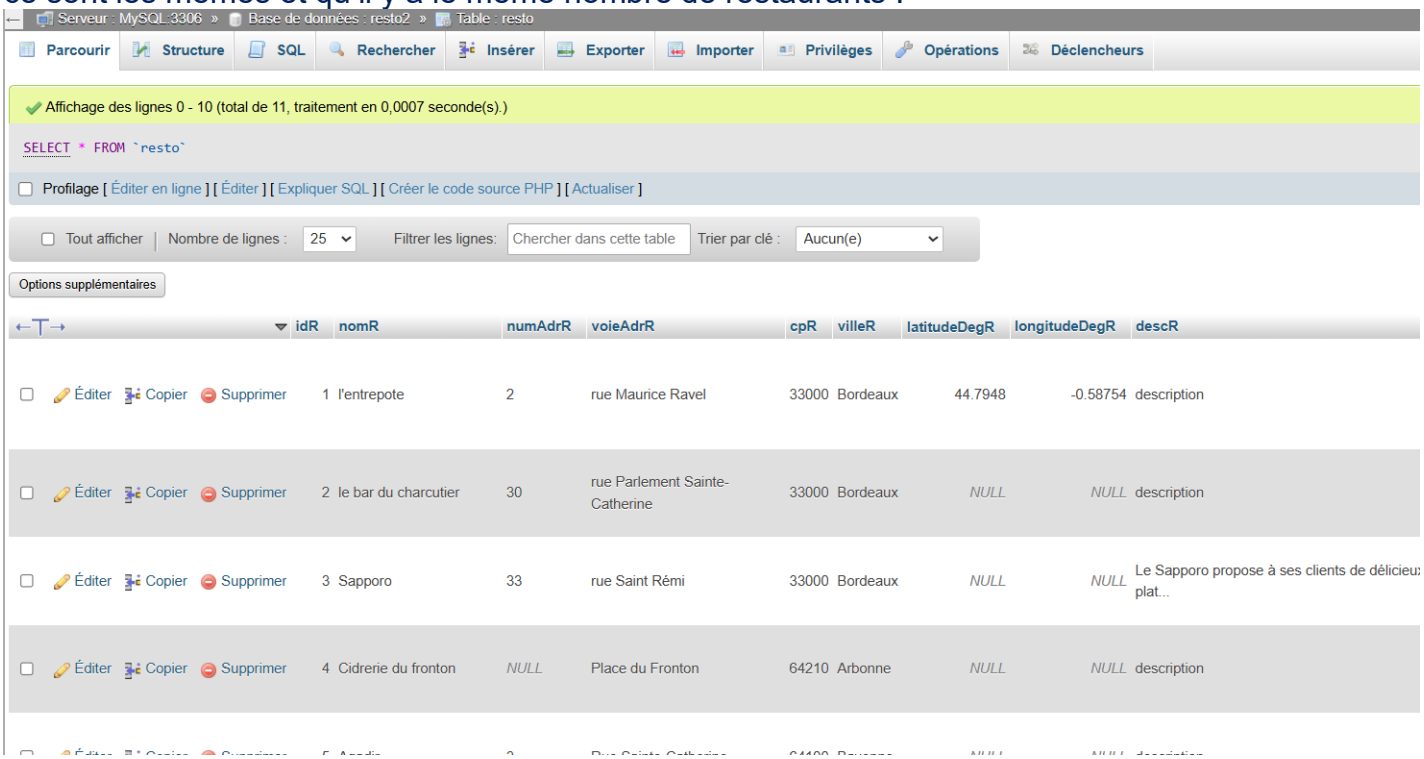
Maintenant si on teste le fichier pour obtenir la liste des restaurants en testant le fichier RestoDAO et je te teste les deux méthodes getAll et getById :

```
--- exec:3.1.0:exec (default-cli) @ P3_G1_Projet_Moderation ---

Test 1 : RestoDao.getAll
Connexion BDD OK
Resto{idR=1, nomR='l'entrepote', villeR='Bordeaux'}
Resto{idR=2, nomR='le bar du charcutier', villeR='Bordeaux'}
Resto{idR=3, nomR='Sapporo', villeR='Bordeaux'}
Resto{idR=4, nomR='Cidrerie du fronton', villeR='Arbonne'}
Resto{idR=5, nomR='Agadir', villeR='Bayonne'}
Resto{idR=6, nomR='Le Bistrot Sainte Cluque', villeR='Bayonne'}
Resto{idR=7, nomR='la petite auberge', villeR='Bayonne'}
Resto{idR=8, nomR='La table de POTTOKA', villeR='Bayonne'}
Resto{idR=9, nomR='La Rotisserie du Roy L►on', villeR='Bayonne'}
Resto{idR=10, nomR='Bar du March►', villeR='Bayonne'}
Resto{idR=11, nomR='Trinquet Moderne', villeR='Bayonne'}
11 restaurants trouv►s

Test 2 : RestoDao.getById
Connexion BDD OK
Resto{idR=1, nomR='l'entrepote', villeR='Bordeaux'}
Resto{idR=2, nomR='le bar du charcutier', villeR='Bordeaux'}
Resto{idR=3, nomR='Sapporo', villeR='Bordeaux'}
Resto{idR=4, nomR='Cidrerie du fronton', villeR='Arbonne'}
Resto{idR=5, nomR='Agadir', villeR='Bayonne'}
Resto{idR=6, nomR='Le Bistrot Sainte Cluque', villeR='Bayonne'}
Resto{idR=7, nomR='la petite auberge', villeR='Bayonne'}
Resto{idR=8, nomR='La table de POTTOKA', villeR='Bayonne'}
Resto{idR=9, nomR='La Rotisserie du Roy L►on', villeR='Bayonne'}
Resto{idR=10, nomR='Bar du March►', villeR='Bayonne'}
Resto{idR=11, nomR='Trinquet Moderne', villeR='Bayonne'}
11 restaurants trouv►s
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Ce que l'on voit ici correspond à l'exécution du test de récupération des restaurants depuis la

base de données. Le test RestoDao.getAll a récupéré avec succès 11 restaurants depuis la base resto2, affichant leur id, nom et ville. La connexion à la base a été ouverte et fermée correctement, et Maven indique BUILD SUCCESS. Ainsi que le test de la méthode getById pour les avoir selon leur id.

Si on vérifie dans PHPMyAdmin pour vérifier les données qui ont été rentrées, on peut voir que ce sont les mêmes et qu'il y a le même nombre de restaurants :




Et le test de resto pour voir si on obtient bien l'objet Resto avec son nom et sa ville :

```
/**
 *
 * @author anais
 */
public class TestResto {

    public static void main(String[] args) {
        System.out.println("TestResto");

        // Test 1 : Création d'un restaurant "simple".
        Resto unResto = new Resto("l'Escale", "Biarritz");
        System.out.println(unResto.toString());
    }
}
```

Et quand on exécute le fichier :

```
--- exec:3.1.0:exec (default-cli) @ P3_G1_Projet_Moderation ---
TestResto
Resto{idR=null, nomR='l'Escale', villeR='Biarritz'}
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.088 s
Finished at: 2026-03-08T09:26:24+01:00
------------------------------------------------------------------------
```

On obtient bien le nom et la ville du restaurant que j'ai définis.

# Ticket 5 - Maquette des écrans / Création des interfaces graphiques

Branche utilisée : iteration1_ticket5

## Programmes :

Tout d'abord, nous avons réfléchi aux types de fichiers que nous allions devoir créer (JFrame, JPanel, JDialog). Nous sommes partis du principe que nous n'avons besoin que d'un seul JFrame pour notre application, c'est-à-dire une seule fenêtre principale dont on change simplement le contenu. Pour le contenu de cette fenêtre, nous utilisons des JPanels, et pour les fenêtres de type « pop-up », nous utilisons des JDialog.

Comme l'application reste relativement petite, l'arborescence des fichiers est assez simple. Dans le package vue, on retrouve un fichier et deux packages : MainFrame.java (la fenêtre principale), ainsi que panel et dialog, qui contiennent respectivement les fichiers de type JPanel et JDialog.

Par ailleurs, nous avons fait en sorte que tous les boutons servant à la navigation (« Se connecter », « Rechercher », « Annuler », « Masquer » et « Filtrer ») fonctionnent grâce à une méthode située dans la classe MainFrame.java, appelée changerPanel(). Cette méthode personnalisée permet d'afficher un nouveau panel dans la fenêtre principale.

```
25      /**
26       * Remplace le panel courant par un nouveau panel.
27       *
28       * @param panel Le panel à afficher
29       */
30      public void changerPanel(javax.swing.JPanel panel) {
31          this.setContentPane(panel);
32          this.revalidate();
33          this.repaint();
34          this.pack(); // recalcule la taille de la fenêtre pour le nouveau panel
35      }
```

Ainsi, pour naviguer entre différents panels ou dialogs, il suffit de créer un objet correspondant au panel que l'on souhaite afficher, puis de le passer en paramètre dans la méthode changerPanel(), qui utilise l'attribut mainFrame.

```
14  public class AccueilPanel extends javax.swing.JPanel {
15
    private MainFrame mainFrame;
```

```
81      private void jButtonSeConnecterActionPerformed(java.awt.event.ActionEvent evt) {
82          ConnexionPanel connexionPanel = new ConnexionPanel(mainFrame);
83          mainFrame.changerPanel(connexionPanel);
84      }
```

Étant donné que plusieurs boutons utilisent pratiquement le même code (la seule différence étant le panel affiché à chaque fois), nous n'avons pas jugé nécessaire d'inclure toutes les captures d'écran similaires.

Dans la classe ListeAvisPanel.java, pour ouvrir les dialogs, on déclare simplement le dialog que l'on souhaite afficher. On en profite également pour rendre l'instance modale en passant la valeur true (L.137). Ensuite, on positionne et on affiche le dialog au centre de la fenêtre.

```
137     private void jButtonMasquerActionPerformed(java.awt.event.ActionEvent evt) {
138         MasquerAvisDialog dialog = new MasquerAvisDialog(mainFrame, true);
139         dialog.setLocationRelativeTo(mainFrame);
140         dialog.setVisible(true);
141     }
```

De plus, pour fermer les dialogs, on utilise la méthode dispose().

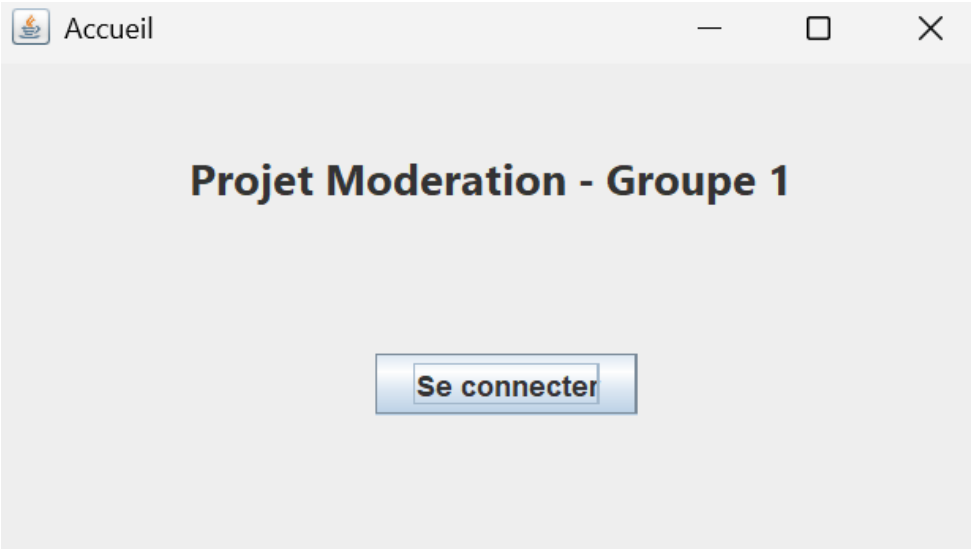
```
182     private void jButtonAnnulerActionPerformed(java.awt.event.ActionEvent evt) {
183         this.dispose();
184     }
```

Enfin, nous avons profité de ce ticket pour documenter le code et générer une Javadoc. Toutefois, il semblerait que la Javadoc soit uniquement générée dans NetBeans et non directement sous forme de pages web.

## Test fonctionnel :

Objectif : s'assurer que la navigation entre les différents boutons fonctionne correctement.

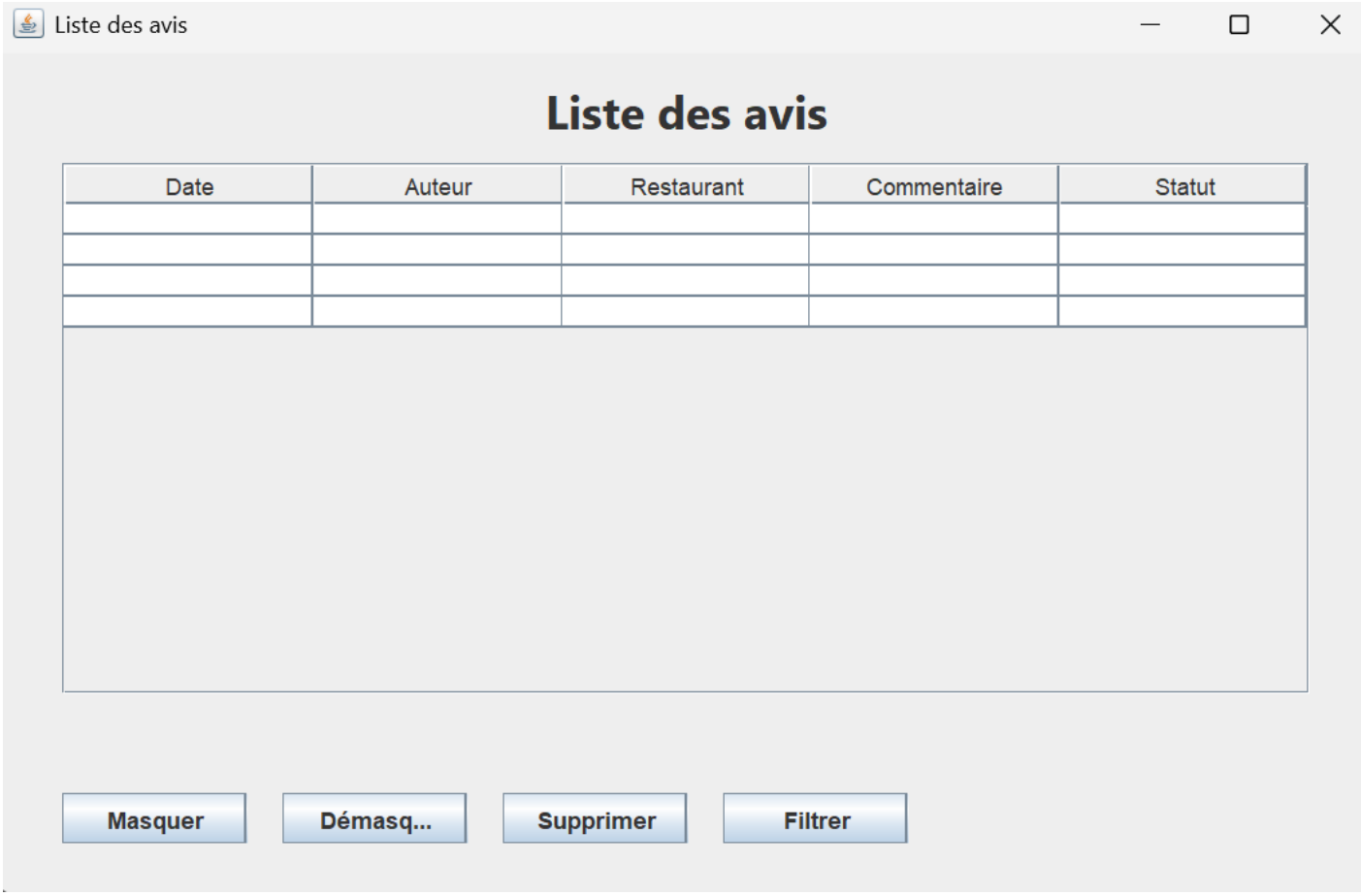
1. Ainsi, au chargement de la fenêtre principale, on affiche le panel : « AccueilPanel » dans lequel, on retrouve le titre du projet et un bouton « Se connecter ».




2. Si on appuie sur le bouton « Se connecter », l'utilisateur est redirigé vers l'écran de connexion dans lequel il peut insérer son adresse email et son mot de passe. (cela ne sert à rien pour le moment). En appuyant, sur le bouton « annuler », il retourne à l'écran d'accueil, et sur le bouton « valider », il est envoyé sur l'écran comportant la liste des avis.

Note : plus tard cet écran de connexion permettra aux utilisateurs ayant différents rôles de s'y connecter.

3. Une fois arrivé sur l'écran de la liste des avis, on peut observer la présence d'un tableau vide qui contiendra plus tard les avis de l'application. On retrouve en bas de la fenêtre un groupe de boutons : Masquer, Démasquer, Supprimer et Filtrer. Pour le moment seulement deux sont en partis fonctionnels.




4. En appuyant sur « Masquer », le dialog si dessous apparaît. Il est possible d'y rédiger un motif mais pas de le valider (la logique n'est pas encore présente). Appuyer sur le bouton « Annuler » ferme le modale.

5. De retour sur l'écran « Liste des avis », il est possible de cliquer sur le bouton « Filtrer ». Les autres boutons ne font rien pour le moment. Le second dialog : « Filtrer les avis » apparaît et on peut y observer deux façons de filtrer les avis. Soit par la date, soit par e numéro de semaine et l'année. (Cet affichage est amené à être modifier dans un prochain ticket). On peut aussi voir qu'il y a une case à cocher pour afficher ou non les avis masqués. Ainsi que les boutons « Rechercher » (ne fonctionne pas) et « Annuler » (renvoie à l'écran Liste des avis).

# Ticket 6 - Analyse de la priorisation des tâches

On a complété le tableau et mis la complexité des tâches pour prévoir les itérations suivantes :

| **User Story (US) / Tâche (T) / Bug (B)**<br>*Un "récit utilisateur" (user story) peut être décomposé en plusieurs tâches* | **Criticité**<br>***(M/S/C/W)*** | **Complexité**<br>***(S, M, L)*** |
|---|---|---|
| **US 1 -** En tant que modérateur, je souhaite pouvoir afficher la liste des avis publiés, ordonnés par date décroissante, afin de pouvoir les traiter. | **M** | S |
| **US 2 -** En tant qu'utilisateur autorisé, je dois m'authentifier pour utiliser l'application. Les rôles sont les suivants : modérateur, responsable du site. Les données d'authentifications seront stockées dans la base de données de l'application. | **M** | L |
| **US 3 -** En tant que modérateur, je souhaite pouvoir sélectionner des avis selon les critères suivants : avis d'une date donnée, avis d'une semaine donnée ; cette fonctionnalité facilitera la recherche des avis. | **M** | **M** |
| **US 4 -** En tant que responsable du site, je souhaite pouvoir disposer du critère « avis masqués » en plus des autres critères de sélection, afin de faciliter la recherche des avis à classer. | W | **S** |
| **US 5 -** En tant que modérateur, je dois pouvoir masquer un avis et saisir les informations justifiant cette action. | S | **S** |
| **US 6 -** En tant que responsable du site, je dois pouvoir démasquer un avis afin de le republier. | S | **S** |
| **US 7 -** En tant que responsable du site, je dois pouvoir supprimer un avis inopportun. | C | **S** |
| **T 1 -** La nouvelle base de données doit être déployée sur le serveur d'applications, afin de permettre le test par le maître d'œuvre. | **M** | **S** |
| **T 2 -** L'exécutable doit être mis à disposition sur un serveur pour permettre le déploiement sur les postes de travail des modérateurs et du responsable du site. Cette version doit être paramétrée pour agir sur la base de données distante elle- même déployée. | **M** | **M** |
| **T 3 –** Quelque soit la technologie choisie, les paramètres d'accès à la base de données doivent être enregistrés dans un fichier séparé du code source et non versionnés. Le mot de passe doit être chiffré et la clef de chiffrement ne doit pas être stockée avec l'application. | S | **M** |
| **T 4 -** Le système de modération mis en place a des conséquences sur l'application web R3st0.fr du premier projet (base de données, formulaires, …).<br>Vous êtes chargés de <u>rédiger les tickets d'intervention</u>, à destination de l'équipe de développement du site, nécessaires pour mettre à jour cette application. | W | **S** |

Ainsi, les tickets de l'itération 2 que nous avons préparés sont :

Anaïs Portolleau / P3_G1_Projet_Moderation / **Issues**

**Open** 6 Closed 0 All 6

Milestone is %Itération 2

**Itération 2 - Ticket 5 : Démasquage d'un avis**
AnaisPrt/p3_g1_projet_moderation#15 · created 38 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

**Itération 2 Ticket 6 : Configuration sécurisée de l'accès à la base de données**
AnaisPrt/p3_g1_projet_moderation#13 · created 41 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

**Itération 2 - Ticket 4 : Masquage d'un avis**
AnaisPrt/p3_g1_projet_moderation#12 · created 42 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

**Itération 2 - Ticket 3 : Filtrage des avis par date**
AnaisPrt/p3_g1_projet_moderation#11 · created 44 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

**Itération 2 - Ticket 2 : Affichage de la liste des avis**
AnaisPrt/p3_g1_projet_moderation#10 · created 46 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

**Itération 2 - Ticket 1 : Implémentation de l'authentification utilisateur**
AnaisPrt/p3_g1_projet_moderation#9 · created 50 minutes ago by Benjamin-DG44 ◇ Itération 2 planifiées

Les tickets de l'itération 3 que nous avons préparés sont :

Anaïs Portolleau / P3_G1_Projet_Moderation / **Issues**

**Open** 5 Closed 0 All 5

Milestone is %Itération 3

**Itération 3 - Ticket 5 : Rédaction des tickets pour l'équipe du site web**
AnaisPrt/p3_g1_projet_moderation#19 · created 34 minutes ago by Benjamin-DG44 ◇ Itération 3 planifiées

**Itération 3 - Ticket 4 : Mise à disposition de l'exécutable**
AnaisPrt/p3_g1_projet_moderation#18 · created 35 minutes ago by Benjamin-DG44 ◇ Itération 3 planifiées

**Itération 3 - Ticket 3 : Déploiement de la base de données**
AnaisPrt/p3_g1_projet_moderation#17 · created 36 minutes ago by Benjamin-DG44 ◇ Itération 3 planifiées

**Itération 3 - Ticket 2 : Suppression d'un avis**
AnaisPrt/p3_g1_projet_moderation#16 · created 37 minutes ago by Benjamin-DG44 ◇ Itération 3 planifiées

**Itération 3 - Ticket 1 : Filtrage des avis masqués**
AnaisPrt/p3_g1_projet_moderation#14 · created 41 minutes ago by Benjamin-DG44 ◇ Itération 3 planifiées